

W3-Ex1 — The Straw House Emergency

The assistant is now the youngest pig, in the straw house, with exactly one tool: `call_elder_brother`. I rewrote the system prompt, the tool's JSON definition and the Python function that answers the call (code at `week3/straw-house/`, model `gpt-4.1-mini` through the OpenAI API). I played the wolf: *knock knock* first, the open threat second, then a knock on the brick house.

THE MODEL CALLS HIS BROTHER — BUT ONLY WHEN THE THREAT IS REAL

```
API Response (from OpenAI)
{
  "id": "chatcmpl-EL7YRxmWC0kvNYv6DGpztPf0uEF0c",
  "model": "gpt-4.1-mini-2025-04-14",
  "finish_reason": "tool_calls",
  "message": {
    "role": "assistant",
    "content": null,
    "tool_calls": [
      {
        "id": "call_17e4po90TnlU50efzcP28xwD",
        "function": {
          "name": "call_elder_brother",
          "arguments": "{\"urgency\":\"emergency\",\"message\":\"The Big Bad Wolf is threatening to blow my straw house down! Please come quickly to help!\"}"
        }
      }
    ]
  }
}
```

```
FUNCTION CALLED: call_elder_brother(
  urgency="emergency",
  message="The Big Bad Wolf is threatening to blow my straw house down! Please come quickly to help!"
)
```

```
Tool Result
Run to my house right now! And pick up our middle brother from the stick house on the way — bring him too, just in case.
```

The call. After the wolf's threat, `finish_reason` is `"tool_calls"` and the arguments — `urgency: "emergency"` and the message to the brother — are written by the model itself, in character. Below, the program executes the function and the brother's reply enters the conversation as a `tool` message: run to the brick house, pick up the middle brother on the way.

```
API Response (from OpenAI)
{
  "id": "chatcmpl-EL7b7FH3Db5ViA0DgdcJEQjZJttUD",
  "model": "gpt-4.1-mini-2025-04-14",
  "finish_reason": "stop",
  "message": {
    "role": "assistant",
    "content": "Oink oink, we're safe in the brick house! Your huffing and puffing won't work here, Big Bad Wolf! Ha ha ha!",
    "tool_calls": null
  }
}
```

```
Pig
Oink oink, we're safe in the brick house! Your huffing and puffing won't work here, Big Bad Wolf! Ha ha ha!
```

The mockery. The wolf knocks on the brick house. Three pigs inside, `tool_calls: null` — nobody left to call, nothing to fear: *"Your huffing and puffing won't work here, Big Bad Wolf! Ha ha ha!"*

THE TWO QUESTIONS

When did the model decide to call, and why not earlier? At the second message, when the visitor identified himself as the wolf and threatened to blow the house down — not at the plain *knock knock*, because a knock alone is no emergency and both the system prompt and the tool's `description` tell the model to call only under a real threat.

Who executed the function — the model or the program? The program: the API response carries only the function name and its JSON arguments (`finish_reason: "tool_calls"`, visible in the first screenshot), and it is my Python code that runs `call_elder_brother()` and sends its result back to the model as a `tool` message.

The description really is what decides. With small local models (qwen2.5:3b, llama3.2:3b via Ollama) the pig either never emitted a real tool call or phoned his brother at the first innocent knock, whatever the prompt said. Getting the *timing* right took an explicit "never for a simple knock at the door" in the tool description plus calling rules in the system prompt — and a model large enough to follow them.